GMoney E-Wallet Monolith

UML Class Diagram

MCO1 Deliverable 2 - GCash FinTech Engineer Specialist Course

Richard Limtin - October 29, 2026

Contents

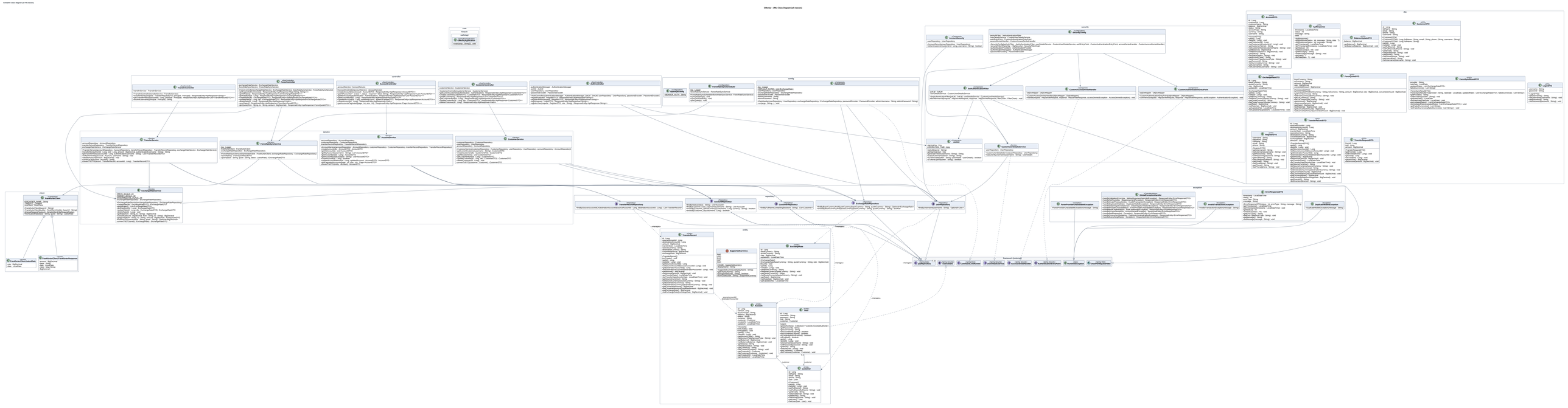
Page 2 Complete class diagram (all 49 classes)
Page 3 1. Domain model: entities and repositories
Page 4 2. Authentication and authorization
Page 5 3. Customers and wallets
Page 6 4. Transfers and ledger
Page 7 5. Forex: rates, live sync and quotes
Page 8 6. Errors, responses and configuration

Notation

+ public - private # protected ~ package-private; underlined members are static.
Solid line, hollow triangle: inheritance (extends). Dashed line, hollow triangle: implements an interface.
Solid arrow with multiplicities: JPA association (Account 0..* -> 1 Customer; User 0..1 -- 1 Customer).
Solid arrow without labels: dependency injected through the constructor (for example TransferService -> AccountRepository).
Dashed arrow <<manages>>: the Spring Data repository that stores that entity. Grey 'framework' boxes are Spring/Java types.
Generated from src/main/java and checked against the compiled classes: every field, method and constructor is shown.

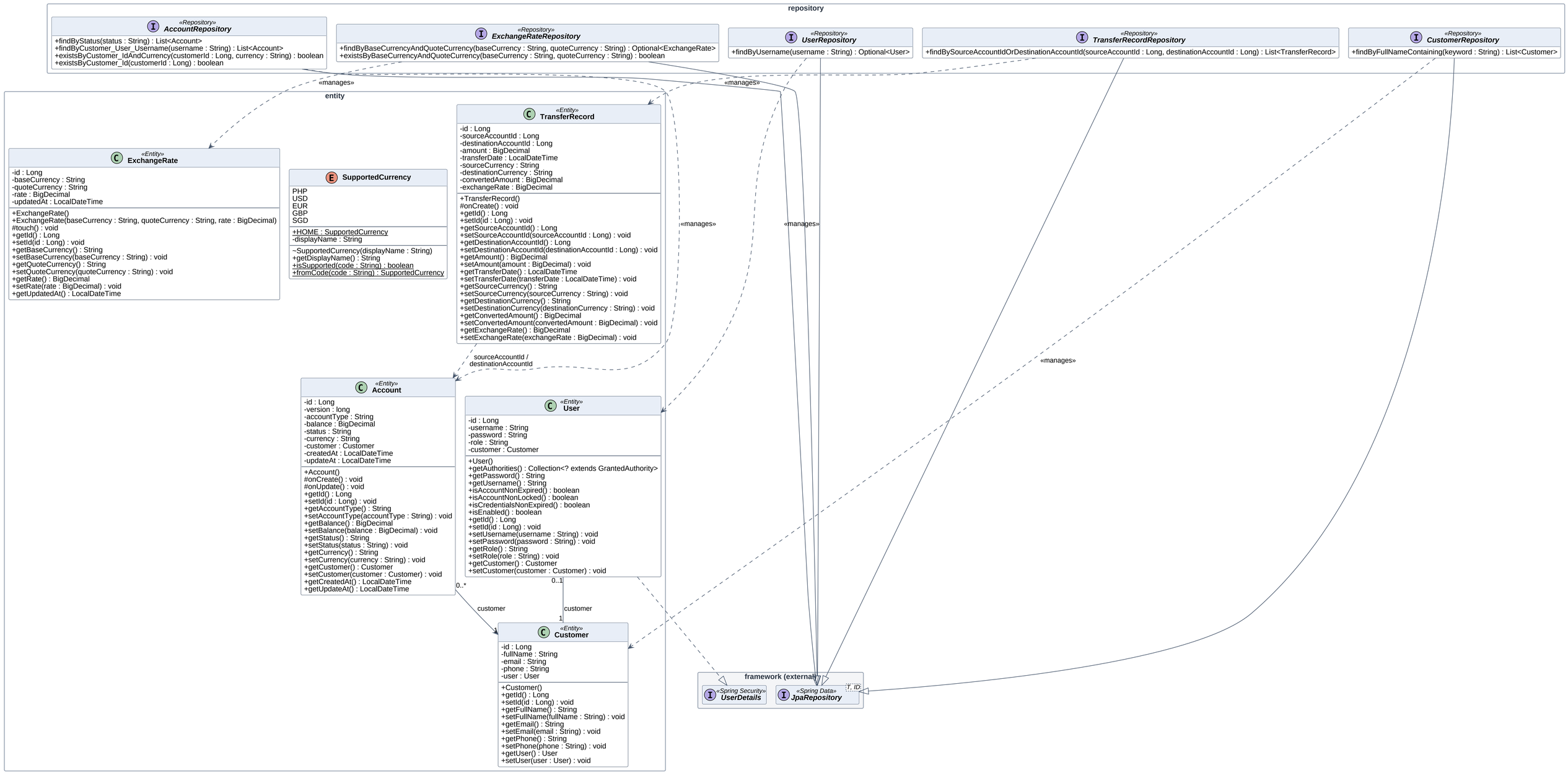
Complete class diagram (all 48 classes).

Odissey - UML Class Diagram (all classes)

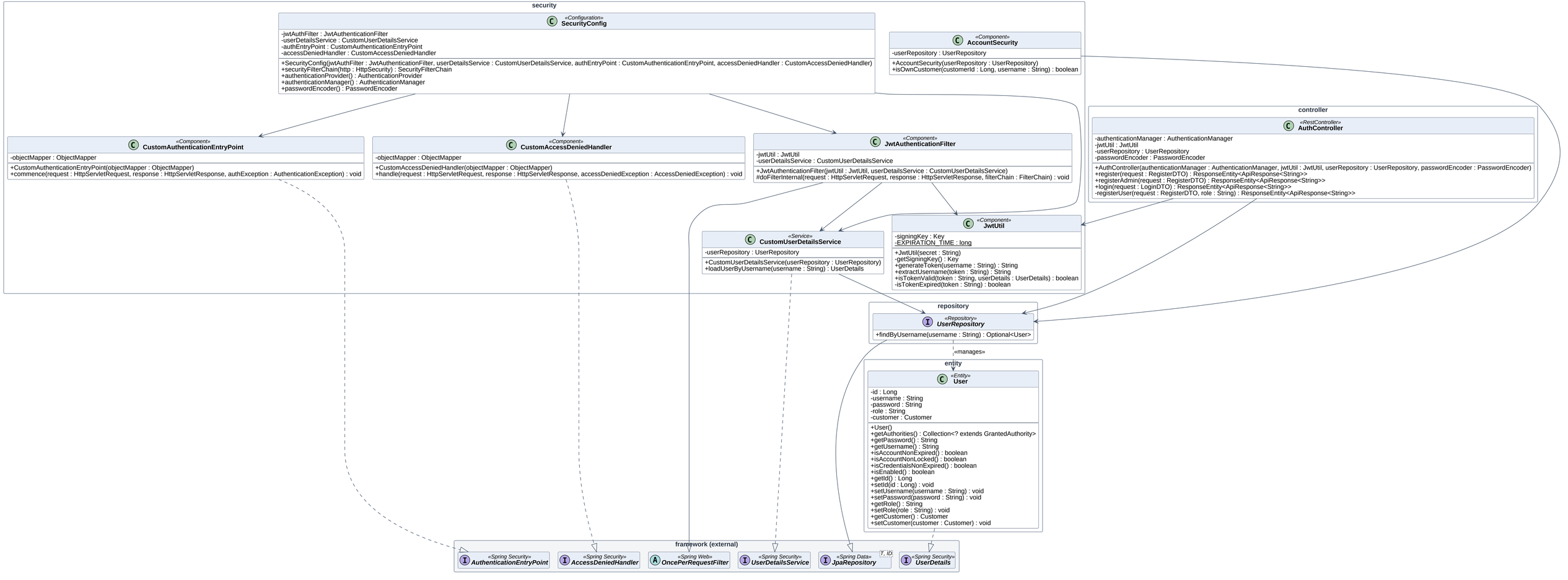

com
fiskoi
authsvgi
«interface»
IAuthentication
+ marcaga : String[] , void
client
«interface»
FraudDetectionClient
FRAUDCHECK_NAME : String
FRAUDCHECK_PATH : String
mvcClient : RestClient
+ FraudDetectionClient() : String
+ FraudDetectionClient(baseUrl : String)
- fromResponse(response : ResponseEntity<String>) : String
- fromClientException(body : String, ex : RestClientException) : String
«record»
FraudCheckResponse
+ isFraudster : boolean
+ reason : String
«record»
TransferCheckHttpClientRequest
+ amount : BigDecimal
+ sourceIban : String
+ targetIban : String
«record»
TransferCheckResponse
+ isFraudster : boolean
+ reason : String
«interface»
TransferCheckClient
+ check(request : TransferCheckHttpClientRequest) : TransferCheckResponse
«interface»
IAccountService
+ createAccount(request : CreateAccountRequest) : AccountDTO
+ getAccountById(id : Long) : AccountDTO
+ getAccountByIban(iban : String) : AccountDTO
+ getAccountsByCustomerId(customerId : Long) : List<AccountDTO>
+ getAllAccounts(pageable : Pageable) : Page<AccountDTO>
+ updateAccount(id : Long, request : UpdateAccountRequest) : AccountDTO
+ deleteAccount(id : Long) : void
+ activateAccount(id : Long) : void
+ deactivateAccount(id : Long) : void
+ freezeAccount(id : Long) : void
+ unfreezeAccount(id : Long) : void
+ closeAccount(id : Long) : void
+ getAccountBalance(id : Long) : BigDecimal
+ hasSufficientBalance(id : Long, amount : BigDecimal) : boolean
+ debitAccount(id : Long, amount : BigDecimal) : void
+ creditAccount(id : Long, amount : BigDecimal) : void
«enum»
AccountStatus
ACTIVE
FROZEN
CLOSED
«enum»
AccountType
CHECKING
SAVINGS
BUSINESS
«record»
CreateAccountRequest
+ customerId : Long
+ accountType : AccountType
+ currency : String
«record»
UpdateAccountRequest
+ accountType : AccountType
+ status : AccountStatus
«record»
AccountDTO
+ id : Long
+ accountNumber : String
+ iban : String
+ customerId : Long
+ accountType : AccountType
+ status : AccountStatus
+ balance : BigDecimal
+ currency : String
+ createdAt : LocalDateTime
+ updatedAt : LocalDateTime
«interface»
IFraudDetectionService
+ checkTransfer(request : FraudCheckRequest) : FraudCheckResponse
«interface»
IFraudHttpClient
+ check(request : FraudCheckRequest) : FraudCheckResponse
«record»
FraudCheckRequest
+ sourceIban : String
+ targetIban : String
+ amount : BigDecimal
+ currency : String
«record»
FraudCheckResponse
+ isFraudster : boolean
+ reason : String
«interface»
ITransferService
+ initiateTransfer(request : TransferRequestDTO) : TransferResponseDTO
+ getTransferById(id : Long) : TransferResponseDTO
+ getTransferByReference(ref : String) : TransferResponseDTO
+ getTransfersByAccount(accountId : Long, pageable : Pageable) : Page<TransferResponseDTO>
+ cancelTransfer(id : Long) : void
«enum»
TransferStatus
PENDING
COMPLETED
FAILED
CANCELLED
«record»
TransferRequestDTO
+ sourceIban : String
+ targetIban : String
+ amount : BigDecimal
+ currency : String
+ description : String
«record»
TransferResponseDTO
+ id : Long
+ reference : String
+ sourceIban : String
+ targetIban : String
+ amount : BigDecimal
+ currency : String
+ status : TransferStatus
+ description : String
+ failureReason : String
+ createdAt : LocalDateTime
+ completedAt : LocalDateTime
«interface»
ICustomerService
+ createCustomer(request : CreateCustomerRequest) : CustomerDTO
+ getCustomerById(id : Long) : CustomerDTO
+ getCustomerByEmail(email : String) : CustomerDTO
+ getAllCustomers(pageable : Pageable) : Page<CustomerDTO>
+ updateCustomer(id : Long, request : UpdateCustomerRequest) : CustomerDTO
+ deactivateCustomer(id : Long) : void
+ activateCustomer(id : Long) : void
«record»
CreateCustomerRequest
+ firstName : String
+ lastName : String
+ email : String
+ phone : String
«record»
UpdateCustomerRequest
+ firstName : String
+ lastName : String
+ phone : String
«record»
CustomerDTO
+ id : Long
+ firstName : String
+ lastName : String
+ email : String
+ phone : String
+ status : CustomerStatus
+ createdAt : LocalDateTime
«enum»
CustomerStatus
ACTIVE
INACTIVE
«interface»
IExchangeRateService
+ getRate(baseCurrency : String, quoteCurrency : String) : BigDecimal
«interface»
IExchangeRateRepository
+ findLatestRate(baseCurrency : String, quoteCurrency : String) : Optional<ExchangeRate>
«record»
ExchangeRateDTO
+ baseCurrency : String
+ quoteCurrency : String
+ rate : BigDecimal
+ timestamp : LocalDateTime
«interface»
IExchangeService
+ convert(request : CurrencyConversionRequest) : CurrencyConversionResponse
+ getRate(baseCurrency : String, quoteCurrency : String) : BigDecimal
«record»
CurrencyConversionRequest
+ sourceAccountId : Long
+ targetAccountId : Long
+ amount : BigDecimal
«record»
CurrencyConversionResponse
+ sourceAccountId : Long
+ targetAccountId : Long
+ originalAmount : BigDecimal
+ convertedAmount : BigDecimal
+ exchangeRate : BigDecimal
«interface»
IExchangeRateProvider
+ fetchRate(baseCurrency : String, quoteCurrency : String) : BigDecimal
«interface»
IExchangeRateCache
+ getRate(baseCurrency : String, quoteCurrency : String) : Optional<BigDecimal>
+ putRate(baseCurrency : String, quoteCurrency : String, rate : BigDecimal) : void
«interface»
IExchangeRateScheduler
+ refreshRates() : void
«enum»
SupportedCurrency
USD
EUR
GBP
JPY
CHF
AUD
CAD
NZD
«record»
ExchangeRate
+ id : Long
+ baseCurrency : String
+ quoteCurrency : String
+ rate : BigDecimal
+ updatedAt : LocalDateTime
«interface»
IExchangeRateRepository
+ findLatestRate(baseCurrency : String, quoteCurrency : String) : Optional<ExchangeRate>
+ findAllLatestRates(baseCurrency : String) : List<ExchangeRate>
+ findRateHistory(baseCurrency : String, quoteCurrency : String, since : LocalDateTime) : List<ExchangeRate>
«interface»
IExchangeRateApiClient
+ fetchLatestRates(baseCurrency : String) : Map<String, BigDecimal>
«interface»
IExchangeRateCache
+ getRate(baseCurrency : String, quoteCurrency : String) : Optional<BigDecimal>
+ putRate(baseCurrency : String, quoteCurrency : String, rate : BigDecimal) : void
«interface»
IExchangeRateScheduler
+ refreshRates() : void
«interface»
IExchangeRateService
+ getRate(baseCurrency : String, quoteCurrency : String) : BigDecimal

1. Domain model: entities and repositories

GMoney - Domain model: entities and repositories



GMoney - Authentication and authorization



GMoney - Customers and wallets

controller

«RestController» AccountController

- -accountService : AccountService
- +AccountController(accountService : AccountService)
- +createAccount(newAccount : AccountDTO) : ResponseEntity<ApiResponse<AccountDTO>>
- +getAllAccounts("" : (value =, status : required = false) String) : ResponseEntity<ApiResponse<List<AccountDTO>>>
- +getMyAccounts(authentication : Authentication) : ResponseEntity<ApiResponse<List<AccountDTO>>>
- +getAccountById(id : Long) : ResponseEntity<ApiResponse<AccountDTO>>
- +updatedAccount(id : Long, updatedAccount : BalanceUpdateDTO) : ResponseEntity<ApiResponse<AccountDTO>>
- +closeAccount(id : Long) : ResponseEntity<ApiResponse<Void>>
- +getAccountsPaginated(page : int, size : int) : ResponseEntity<ApiResponse<Page<AccountDTO>>>

«RestController» CustomerController

- -customerService : CustomerService
- +CustomerController(customerService : CustomerService)
- +getCurrentCustomer(authentication : Authentication) : ResponseEntity<ApiResponse<CustomerDTO>>
- +createCustomer(customerDTO : CustomerDTO) : ResponseEntity<ApiResponse<CustomerDTO>>
- +getAllCustomers() : ResponseEntity<ApiResponse<List<CustomerDTO>>>
- +getCustomerById(id : Long) : ResponseEntity<ApiResponse<CustomerDTO>>
- +updateCustomer(id : Long, dto : CustomerDTO) : ResponseEntity<ApiResponse<CustomerDTO>>
- +deleteCustomer(id : Long) : ResponseEntity<ApiResponse<Void>>

service

«Service» AccountService

- -accountRepository : AccountRepository
- -customerRepository : CustomerRepository
- -transferRecordRepository : TransferRecordRepository
- +AccountService(accountRepository : AccountRepository, customerRepository : CustomerRepository, transferRecordRepository : TransferRecordRepository)
- +createAccount(dto : AccountDTO) : AccountDTO
- +getAllAccounts() : List<AccountDTO>
- +getMyAccounts(username : String) : List<AccountDTO>
- +getAccountById(id : Long) : AccountDTO
- +getAccountByStatus(status : String) : List<AccountDTO>
- +closeAccount(id : Long) : boolean
- +updateAccountBalance(id : Long, updatedAccount : AccountDTO) : AccountDTO
- +getPaginatedAccounts(page : int, size : int) : Page<AccountDTO>
- -convertToDTO(entity : Account) : AccountDTO

«Service» CustomerService

- -customerRepository : CustomerRepository
- -userRepository : UserRepository
- -accountRepository : AccountRepository
- +CustomerService(customerRepository : CustomerRepository, userRepository : UserRepository, accountRepository : AccountRepository)
- +getCurrentCustomer(username : String) : CustomerDTO
- +createCustomer(dto : CustomerDTO) : CustomerDTO
- +getAllCustomers() : List<CustomerDTO>
- +getCustomerById(id : Long) : CustomerDTO
- +updateCustomer(id : Long, dto : CustomerDTO) : CustomerDTO
- +deleteCustomer(id : Long) : void
- -convertToDTO(customer : Customer) : CustomerDTO

security

«Component» AccountSecurity

- -userRepository : UserRepository
- +AccountSecurity(userRepository : UserRepository)
- +isOwnCustomer(customerId : Long, username : String) : boolean

dto

«DTO» AccountDTO

- -id : Long
- -customerId : Long
- -customerName : String
- -balance : BigDecimal
- -status : String
- -accountType : String
- -currency : String
- -username : String
- +AccountDTO()
- +getId() : Long
- +setId(id : Long) : void
- +getCustomerId() : Long
- +setCustomerId(customerId : Long) : void
- +getCustomerName() : String
- +setCustomerName(customerName : String) : void
- +getBalance() : BigDecimal
- +setBalance(balance : BigDecimal) : void
- +getStatus() : String
- +setStatus(status : String) : void
- +getAccountType() : String
- +setAccountType(accountType : String) : void
- +getCurrency() : String
- +setCurrency(currency : String) : void
- +getUsername() : String
- +setUsername(username : String) : void

«DTO» BalanceUpdateDTO

- -balance : BigDecimal
- +getBalance() : BigDecimal
- +setBalance(balance : BigDecimal) : void

«DTO» CustomerDTO

- -id : Long
- -fullName : String
- -email : String
- -phone : String
- -username : String
- +CustomerDTO()
- +CustomerDTO(id : Long, fullName : String, email : String, phone : String, username : String)
- +CustomerDTO(id : Long, fullName : String)
- +getId() : Long
- +setId(id : Long) : void
- +getFullName() : String
- +setFullName(fullName : String) : void
- +getEmail() : String
- +setEmail(email : String) : void
- +getPhone() : String
- +setPhone(phone : String) : void
- +getUsername() : String
- +setUsername(username : String) : void

repository

«Repository» CustomerRepository

- +findByFullNameContaining(keyword : String) : List<Customer>

«Repository» TransferRecordRepository

- +findBySourceAccountIdOrDestinationAccountId(sourceAccountId : Long, destinationAccountId : Long) : List<TransferRecord>

«Repository» AccountRepository

- +findByStatus(status : String) : List<Account>
- +findByCustomer_User_Username(username : String) : List<Account>
- +existsByCustomer_IdAndCurrency(customerId : Long, currency : String) : boolean
- +existsByCustomer_Id(customerId : Long) : boolean

«Repository» UserRepository

- +findByUsername(username : String) : Optional<User>

«manages»

«manages»

«manages»

entity

«Entity» Account

- -id : Long
- -version : long
- -accountType : String
- -balance : BigDecimal
- -status : String
- -currency : String
- -customer : Customer
- -createdAt : LocalDateTime
- -updateAt : LocalDateTime
- +Account()
- #onCreate() : void
- #onUpdate() : void
- +getId() : Long
- +setId(id : Long) : void
- +getAccountType() : String
- +setAccountType(accountType : String) : void
- +getBalance() : BigDecimal
- +setBalance(balance : BigDecimal) : void
- +getStatus() : String
- +setStatus(status : String) : void
- +getCurrency() : String
- +setCurrency(currency : String) : void
- +getCustomer() : Customer
- +setCustomer(customer : Customer) : void
- +getCreatedAt() : LocalDateTime
- +getUpdateAt() : LocalDateTime

«Entity» User

- -id : Long
- -username : String
- -password : String
- -role : String
- -customer : Customer
- +User()
- +getAuthorities() : Collection<? extends GrantedAuthority>
- +getPassword() : String
- +getUsername() : String
- +isAccountNonExpired() : boolean
- +isAccountNonLocked() : boolean
- +isCredentialsNonExpired() : boolean
- +isEnabled() : boolean
- +getId() : Long
- +setId(id : Long) : void
- +setUsername(username : String) : void
- +setPassword(password : String) : void
- +getRole() : String
- +setRole(role : String) : void
- +getCustomer() : Customer
- +setCustomer(customer : Customer) : void

0..1

0..*

customer

1

customer

1

«Entity» Customer

- -id : Long
- -fullName : String
- -email : String
- -phone : String
- -user : User
- +Customer()
- +getId() : Long
- +setId(id : Long) : void
- +getFullName() : String
- +setFullName(fullName : String) : void
- +getEmail() : String
- +setEmail(email : String) : void
- +getPhone() : String
- +setPhone(phone : String) : void
- +getUser() : User
- +setUser(user : User) : void

exception

«Exception» DuplicateWalletException

- +DuplicateWalletException(message : String)

framework (external)

«Spring Security» UserDetails

«Spring Data» JpaRepository<T, ID>

«Java» RuntimeException

GMoney - Transfers and ledger

dto

«DTO» TransferRequestDTO

- -fromId : Long
- -toId : Long
- -amount : BigDecimal

+TransferRequestDTO()
+getFromId() : Long
+setFromId(fromId : Long) : void
+getToId() : Long
+setToId(toId : Long) : void
+getAmount() : BigDecimal
+setAmount(amount : BigDecimal) : void

«DTO» TransferRecordDTO

- -id : Long
- -sourceAccountId : Long
- -destinationAccountId : Long
- -amount : BigDecimal
- -transferDate : LocalDateTime
- -sourceCurrency : String
- -destinationCurrency : String
- -convertedAmount : BigDecimal
- -exchangeRate : BigDecimal
- -direction : String

+TransferRecordDTO()
+getId() : Long
+setId(id : Long) : void
+getSourceAccountId() : Long
+setSourceAccountId(sourceAccountId : Long) : void
+getDestinationAccountId() : Long
+setDestinationAccountId(destinationAccountId : Long) : void
+getAmount() : BigDecimal
+setAmount(amount : BigDecimal) : void
+getTransferDate() : LocalDateTime
+setTransferDate(transferDate : LocalDateTime) : void
+getSourceCurrency() : String
+setSourceCurrency(sourceCurrency : String) : void
+getDestinationCurrency() : String
+setDestinationCurrency(destinationCurrency : String) : void
+getConvertedAmount() : BigDecimal
+setConvertedAmount(convertedAmount : BigDecimal) : void
+getExchangeRate() : BigDecimal
+setExchangeRate(exchangeRate : BigDecimal) : void
+getDirection() : String
+setDirection(direction : String) : void

controller

«RestController» TransferController

- -transferService : TransferService

+TransferController(transferService : TransferService)
+transferMoney(request : TransferRequestDTO, principal : Principal) : ResponseEntity<ApiResponse<String>>
+getTransferHistory(accountId : Long, principal : Principal) : ResponseEntity<ApiResponse<List<TransferRecordDTO>>>
-resolveUsername(principal : Principal) : String

service

«Service» TransferService

- -accountRepository : AccountRepository
- -transferRecordRepository : TransferRecordRepository
- -exchangeRateService : ExchangeRateService

+TransferService(accountRepository : AccountRepository, transferRecordRepository : TransferRecordRepository, exchangeRateService : ExchangeRateService)
+transferMoney(fromId : Long, toId : Long, amount : BigDecimal, authenticatedUsername : String) : String
+getTransferHistory(accountId : Long, authenticatedUsername : String) : List<TransferRecordDTO>
-verifyOwnership(account : Account, username : String) : void
-validateAmount(amount : BigDecimal) : void
-currencyOf(account : Account) : String
-convertToDTO(record : TransferRecord, accountId : Long) : TransferRecordDTO

«Service» ExchangeRateService

- +RATE_SCALE : int
- +MONEY_SCALE : int
- -INTERMEDIATE_SCALE : int
- -exchangeRateRepository : ExchangeRateRepository

+ExchangeRateService(exchangeRateRepository : ExchangeRateRepository)
+createRate(dto : ExchangeRateDTO) : ExchangeRateDTO
+getAllRates() : List<ExchangeRateDTO>
+getRateById(id : Long) : ExchangeRateDTO
+updateRate(id : Long, dto : ExchangeRateDTO) : ExchangeRateDTO
+deleteRate(id : Long) : void
+getRate(from : String, to : String) : BigDecimal
+convert(amount : BigDecimal, from : String, to : String) : BigDecimal
+getQuote(from : String, to : String, amount : BigDecimal) : ForexQuoteDTO
-findDirectOrInverse(base : String, quote : String) : Optional<BigDecimal>
-convertToDTO(entity : ExchangeRate) : ExchangeRateDTO

exception

«Exception» InvalidTransactionException

+InvalidTransactionException(message : String)

repository

«Repository» TransferRecordRepository

+findBySourceAccountIdOrDestinationAccountId(sourceAccountId : Long, destinationAccountId : Long) : List<TransferRecord>

«Repository» AccountRepository

+findByStatus(status : String) : List<Account>
+findByCustomer_User_Username(username : String) : List<Account>
+existsByCustomer_IdAndCurrency(customerId : Long, currency : String) : boolean
+existsByCustomer_Id(customerId : Long) : boolean

«manages»

entity

«Entity» TransferRecord

- -id : Long
- -sourceAccountId : Long
- -destinationAccountId : Long
- -amount : BigDecimal
- -transferDate : LocalDateTime
- -sourceCurrency : String
- -destinationCurrency : String
- -convertedAmount : BigDecimal
- -exchangeRate : BigDecimal

+TransferRecord()
#onCreate() : void
+getId() : Long
+setId(id : Long) : void
+getSourceAccountId() : Long
+setSourceAccountId(sourceAccountId : Long) : void
+getDestinationAccountId() : Long
+setDestinationAccountId(destinationAccountId : Long) : void
+getAmount() : BigDecimal
+setAmount(amount : BigDecimal) : void
+getTransferDate() : LocalDateTime
+setTransferDate(transferDate : LocalDateTime) : void
+getSourceCurrency() : String
+setSourceCurrency(sourceCurrency : String) : void
+getDestinationCurrency() : String
+setDestinationCurrency(destinationCurrency : String) : void
+getConvertedAmount() : BigDecimal
+setConvertedAmount(convertedAmount : BigDecimal) : void
+getExchangeRate() : BigDecimal
+setExchangeRate(exchangeRate : BigDecimal) : void

sourceAccountId / destinationAccountId

«manages»

«Entity» Account

- -id : Long
- -version : long
- -accountType : String
- -balance : BigDecimal
- -status : String
- -currency : String
- -customer : Customer
- -createdAt : LocalDateTime
- -updateAt : LocalDateTime

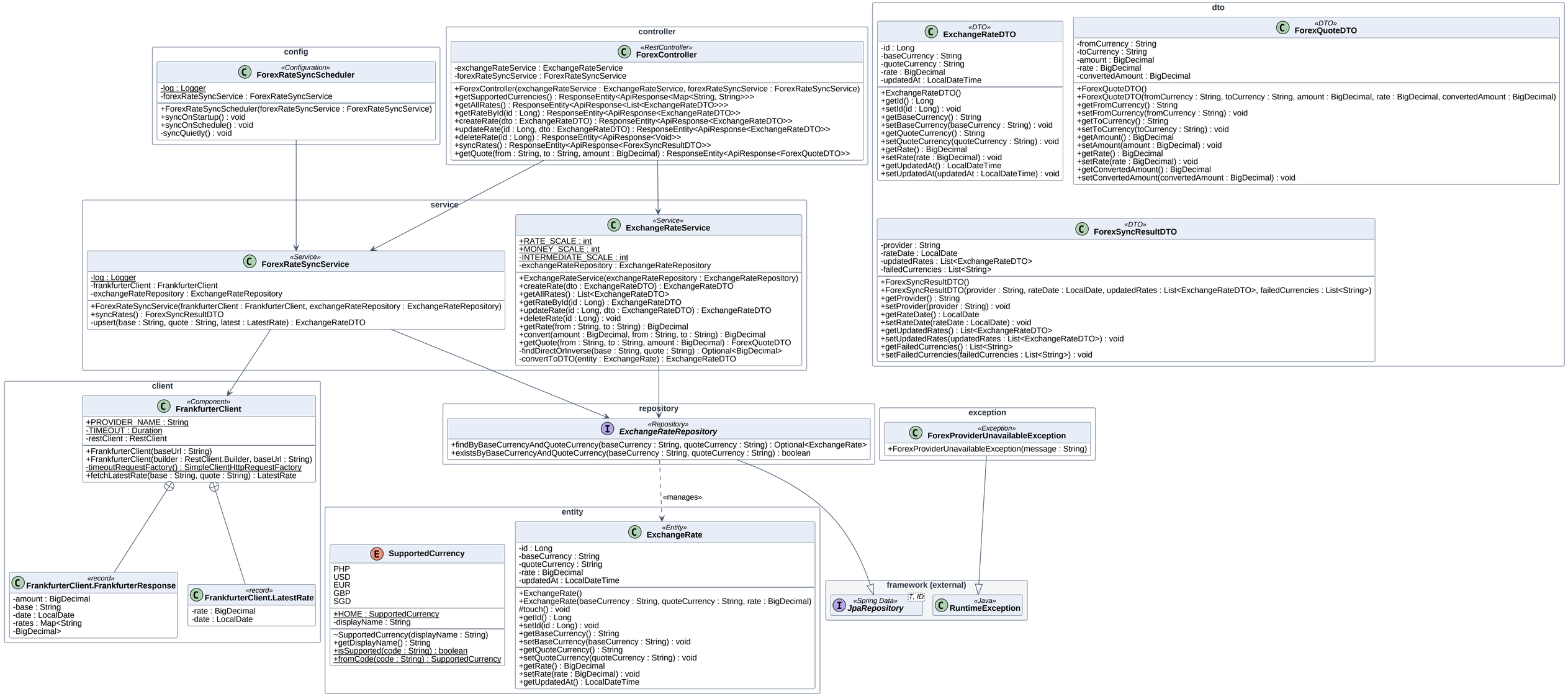
+Account()
#onCreate() : void
#onUpdate() : void
+getId() : Long
+setId(id : Long) : void
+getAccountType() : String
+setAccountType(accountType : String) : void
+getBalance() : BigDecimal
+setBalance(balance : BigDecimal) : void
+getStatus() : String
+setStatus(status : String) : void
+getCurrency() : String
+setCurrency(currency : String) : void
+getCustomer() : Customer
+setCustomer(customer : Customer) : void
+getCreatedAt() : LocalDateTime
+getUpdateAt() : LocalDateTime

framework (external)

«Java» RuntimeException

«Spring Data» JpaRepository <T, ID>

GMoney - Forex: rates, live sync and quotes



GMoney - Errors, responses and configuration

